Transformers

Învățare Automată

Mihai Nan & Adina Magda Florea

Departamentul de Calculatoare
Facultatea de Automatică și Calculatoare
Universitatea Națională de Știință și Tehnologie POLITEHNICA București

Ianuarie 2026

Cuprins

1. Contextual embedding
2. Transformer
3. Large Language Models

Contextual embedding

- **Contextualizarea**: captează sensul unui cuvânt/token în contextul său specific.
- Oferă nuanțele necesare pentru modele de ML, în special în NLP.
- **Embedding-uri contextualizate**: reprezentarea unui cuvânt variază în funcție de cuvintele din jur.

Exemple:

- "Ana citește o carte interesantă."
- "El a câștigat ultima carte la jocul de cărți.""

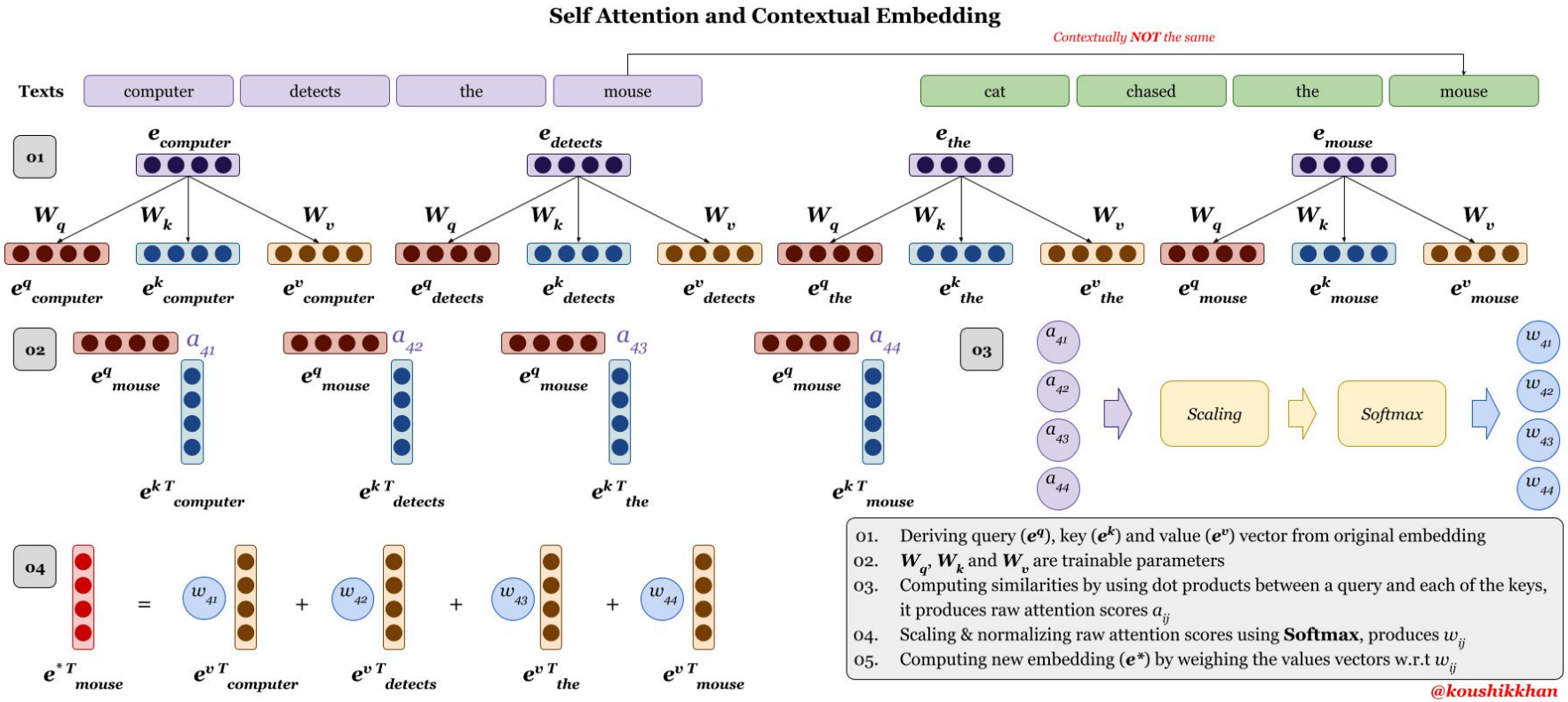
Embedding-uri tradiționale (Word2Vec):

- Cuvântul "carte" are același vector în ambele propoziții.

Contextual embeddings (ex.: BERT):

- În propoziția 1, "carte" $\rightarrow$ sens de *carte literară*.
- În propoziția 2, "carte" $\rightarrow$ sens de *carte de joc*.
- Rezultă **vectori diferiți** pentru același cuvânt în contexte diferite.

Contextual embeddings folosesc mecanismul de self-attention, care permite fiecărui cuvânt să-și ajusteze reprezentarea în funcție de relația cu celelalte cuvinte din propoziție.



Modele Populare pentru Embedding-uri Contextualizate

ELMo (Embeddings from Language Models)

- Generează embedding-uri utilizând întreaga propoziție.
- Folosește LSTM-uri bidirecționale pentru a captura contextul înainte și înapoi.
- Produce embedding-uri care reflectă rolul cuvântului în propoziție.

BERT (Bidirectional Encoder Representations from Transformers)

- Folosește arhitectura Transformer bidirecțională.
- Ia în considerare simultan contextul din stânga și din dreapta.
- Pre-antrenat pe un corpus foarte mare.

GPT (Generative Pre-trained Transformer)

- Model unidirecțional (stânga $\rightarrow$ dreapta).
- Generează embedding-uri bogate contextual prin pre-antrenare masivă.

Contextual embedding – Exemplu

Ana citește o carte bună

Cuvânt	Reprezentare (Embedding)
Ana	$[0.5, 0.8, 0.1]$
Citește	$[0.7, 0.2, 0.9]$
O	$[0.3, 0.1, 0.2]$
Carte	$[0.9, 0.5, 1.2]$
Bună	$[0.6, 0.9, 0.7]$

Proiecțiile pentru **Queries (Q)**, **Keys (K)** și **Values (V)** sunt calculate pentru fiecare cuvânt folosind matrici de W_Q, W_K, W_V

Cuvânt	Q (Query)	K (Key)	V (Value)
Ana	$[0.6, 0.7, 0.2]$	$[0.5, 0.9, 0.3]$	$[0.7, 0.8, 0.4]$
Citește	$[0.8, 0.6, 0.5]$	$[0.7, 0.4, 0.8]$	$[0.6, 0.7, 0.9]$
O	$[0.2, 0.3, 0.1]$	$[0.4, 0.2, 0.3]$	$[0.3, 0.4, 0.2]$
Carte	$[0.9, 0.8, 1.0]$	$[1.0, 0.7, 1.1]$	$[0.9, 0.8, 1.2]$
Bună	$[0.7, 0.5, 0.8]$	$[0.8, 0.6, 0.9]$	$[0.8, 0.9, 0.7]$

Contextual embedding – Exemplu

Folosim produsul scalar QK^T pentru a calcula scorurile de atenție între fiecare Query și toate Keys.

	Ana	Citește	O	Carte	Bună
Ana	1.23	1.01	0.67	1.42	1.11
Citește	1.01	1.45	0.89	1.33	1.29
O	0.67	0.89	0.32	0.78	0.54
Carte	1.42	1.33	0.78	1.67	1.51
Bună	1.11	1.29	0.54	1.51	1.39

Softmax pentru normalizare

	Ana	Citește	O	Carte	Bună
Ana	0.29	0.24	0.13	0.20	0.14
Citește	0.22	0.28	0.16	0.18	0.16
O	0.16	0.22	0.10	0.26	0.16
Carte	0.24	0.20	0.13	0.25	0.18
Bună	0.20	0.26	0.12	0.28	0.14

Contextual embedding – Exemplu

Pentru fiecare cuvânt, combinăm Valorile V ale tuturor cuvintelor folosind ponderile calculate (probabilitățile Softmax)

$$\text{Noua reprezentare pentru Ana} =$$

$$= (0.29 \cdot V_{Ana}) + (0.24 \cdot V_{Citește}) + (0.13 \cdot V_{O}) + (0.20 \cdot V_{Carte}) + (0.14 \cdot V_{Bună})$$

Noile reprezentări ale cuvintelor reflectă relațiile dintre ele:

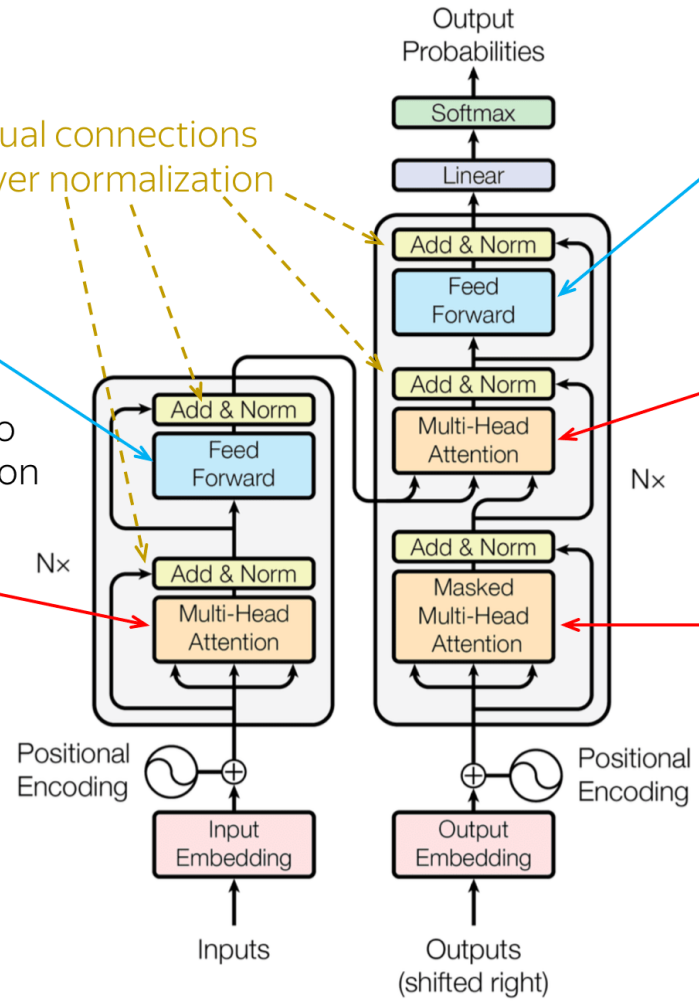
- „Ana” este influențat de „citește” și „carte,” deoarece acestea sunt cuvinte relevante pentru înțelegerea propoziției
- „Bună” are o relație puternică cu „carte,” deoarece este un atribut direct al acesteia

Contextual embedding – Exemplu

Rezultat

Token	Contextual Embedding (vector cu 768 dimensiuni)	
[CLS]	$[0.21, -0.34, 0.45, ..., 0.12, 0.34, -0.56]$	Vector general pentru propoziție.
Ana	$[0.67, -0.12, 0.34, ..., 0.23, -0.45, 0.78]$	Vector contextualizat: Reflectă rolul de subiect.
citește	$[0.12, -0.45, 0.78, ..., -0.11, 0.67, 0.22]$	Vector contextualizat: Reflectă verbul și relațiile sale.
o	$[0.23, -0.11, 0.43, ..., 0.09, 0.56, -0.34]$	Vector contextualizat: Reflectă articolul „o" în relație cu „carte."
carte	$[0.45, 0.67, -0.12, ..., 0.56, -0.22, 0.34]$	Vector contextualizat: Include informații despre adjectivul „bună."
bună	$[0.34, 0.23, -0.11, ..., 0.12, 0.45, 0.67]$	Vector contextualizat: Legat semantic de „carte."
[SEP]	$[-0.11, 0.22, -0.34, ..., 0.45, 0.12, -0.67]$	Vector de separare (informație structurală).

Transformer



Residual connections and layer normalization

Feed-forward network: after taking information from other tokens, take a moment to think and process this information

Encoder self-attention: tokens look at each other

queries, keys, values are computed from encoder states

Feed-forward network: after taking information from other tokens, take a moment to think and process this information

Decoder-encoder attention: target token looks at the source

queries – from decoder states; keys and values from encoder states

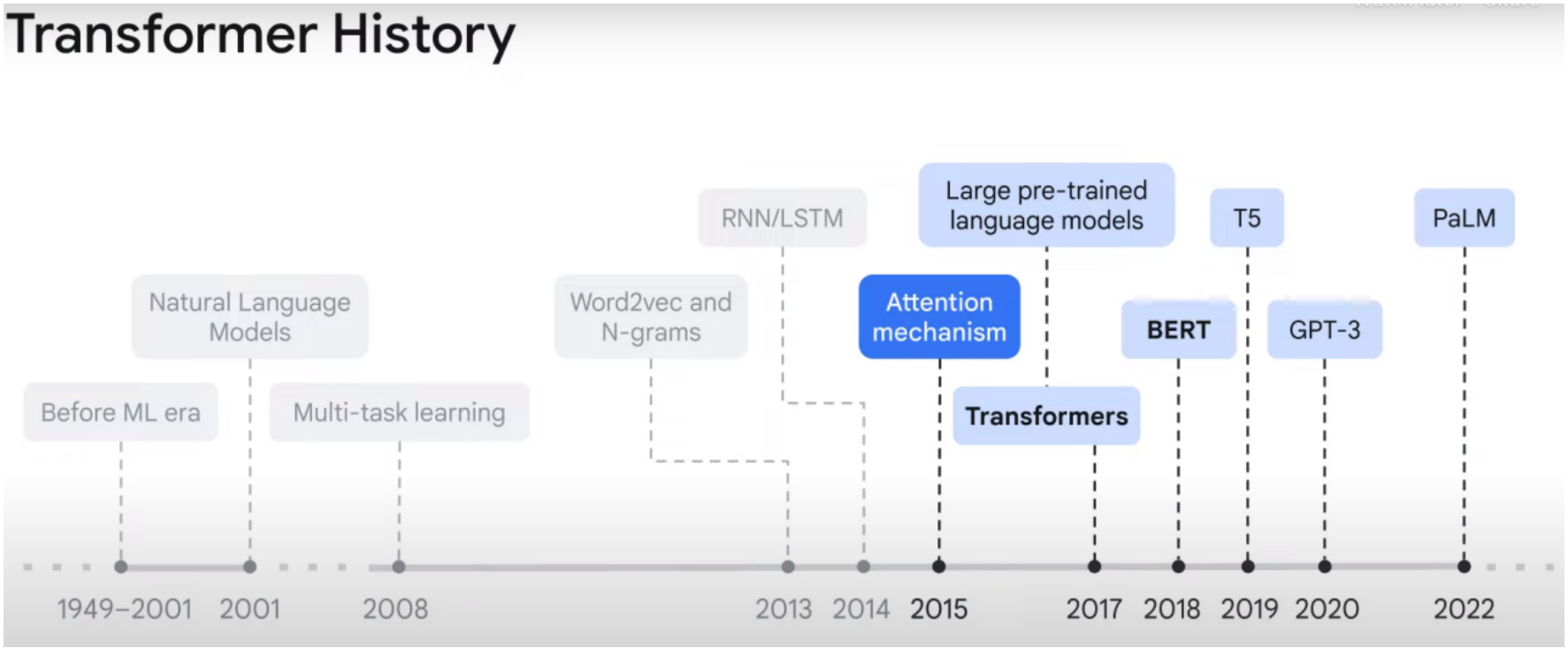
Decoder self-attention (masked): tokens look at the previous tokens

queries, keys, values are computed from decoder states

Transformers

- Introduse inițial pentru traducere automata, acum utilizate larg pentru multiple taskuri.
- Inspirate din strucura de Encoder-Decoder de la RNN dar elimină anumite probleme ale RNN.
- RNN prelucrează secvențial, nu pot exploata capacitatea de prelucrare paralelă (GPU). Prin natura sa, RNN este pur secvențial.
- Astfel, devin ineficiente pentru secvențe și dependențe lungi.
- Combină avantajele CNN (paralelizare) cu cele ale RNN de prelucrare a secvențeleor de date în timp.
- Moment important marcat de lucrarea *Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).*

Transformers – Istoric



Sursă: Coursera - Transformer Models and BERT Model

Puțină istorie: de unde pornește atenția?

- Mulți asociază mecanismul de **atenție** cu lucrarea *"Attention is All You Need"* (Google Brain).
- Totuși, ideea de bază apare mai devreme, în contextul **traducerii automate folosind rețele neurale**.
- Rădăcinile se găsesc în lucrarea: *"Neural Machine Translation by Jointly Learning to Align and Translate"*.
- Procesarea limbajului natural (NLP) implică lucrul cu **secvențe de lungime variabilă**.
- Primele modele performante au fost bazate pe:
  - Rețele Neurale Recurente (RNN)
  - LSTM și GRU
- Aceste modele procesează informația **secvențial**, pas cu pas.
- Limitările lor devin evidente pentru secvențe lungi.

Problema reprezentării propozițiilor lungi

- În modelele clasice encoder–decoder:
  - propoziția sursă este comprimată într-un singur vector
- Pentru propoziții lungi:
  - informația relevantă se pierde
  - dependențele pe termen lung sunt greu de capturat
- Acest fenomen limitează calitatea traducerilor automate.
- Problema este cunoscută drept **information bottleneck**.

Intuiția atenției

- Traducerea umană:
  - revenim constant la propoziția sursă
  - ne concentrăm pe fragmente relevante
- Atenția permite modelului să:
  - *se uite* selectiv la părți diferite ale intrării
  - acorde importanță diferită fiecărui cuvânt
- Rezultatul:
  - mai puțină pierdere de informație
  - traduceri mai corecte și mai fluente

Transformers: Intuiție generală

- În **encoder**, tokenii comunică între ei prin mecanisme de self-attention și își actualizează reprezentările.
- În **decoder**, fiecare token țintă:
  - privește mai întâi tokenii deja generați (self-attention),
  - apoi se uită la reprezentările din encoder (encoder–decoder attention),
  - și își actualizează vectorul-reprezentare.
- Acest proces are loc în mai multe straturi succesive (tipic **6 straturi în encoder** și **6 straturi în decoder**).
- Fiecare strat rafinează înțelegerea relațiilor dintre tokeni și adaugă context suplimentar.

În continuare: analizăm în detaliu celelalte componente ale arhitecturii Transformer.

Transformers: Arhitectura generală

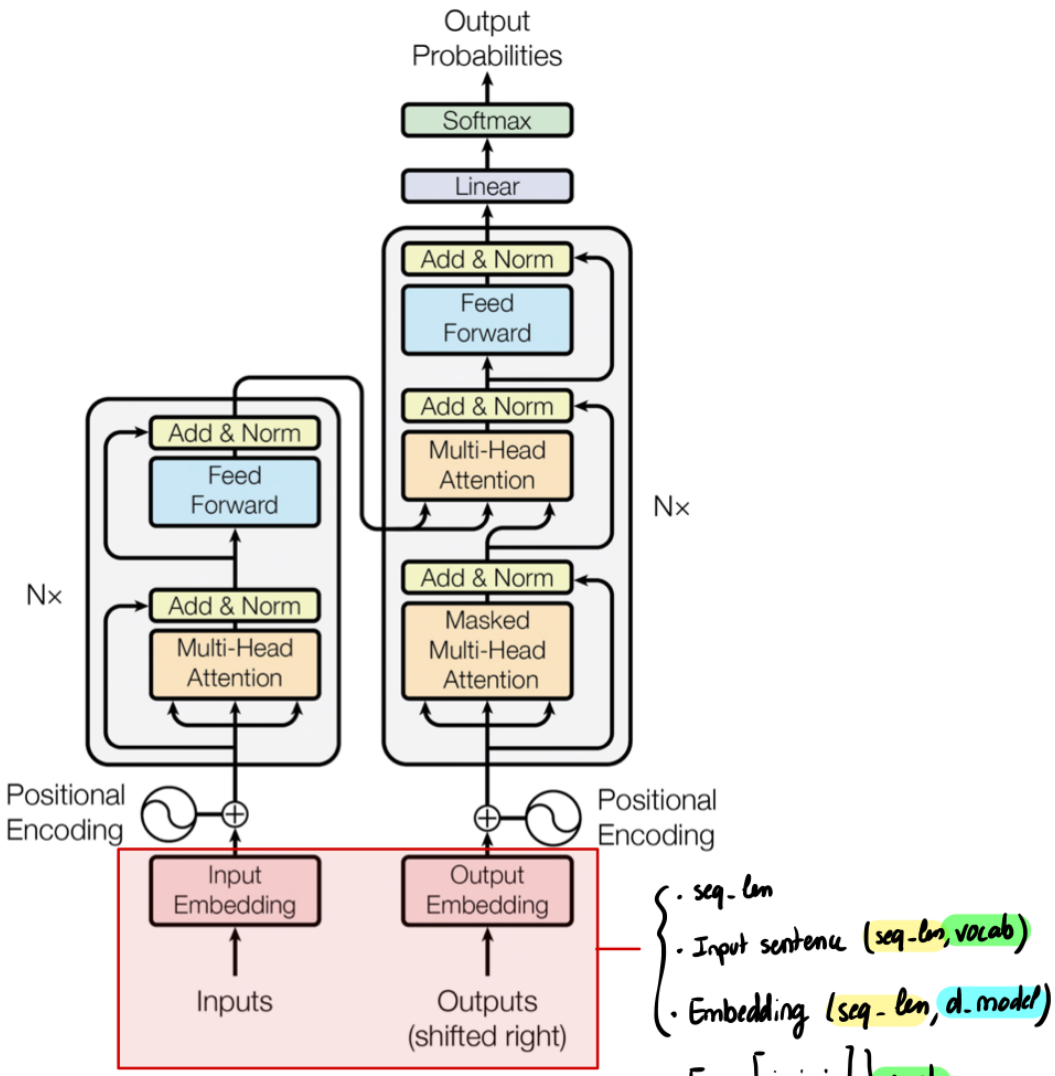


Figure 1: The Transformer - model architecture.

De la text la vectori: embeddings

- Arhitectura Transformer lucrează exclusiv cu reprezentări numerice, nu direct cu text.
- Prin urmare, primul pas esențial este transformarea cuvintelor dintr-o propoziție în vectori numerici. Considerăm o secvență de intrare precum *The cat drinks milk*, având o lungime numită `seq_len`. Fiecare cuvânt din această secvență trebuie convertit într-o formă pe care modelul o poate procesa matematic.
- Acest proces se numește **embedding**. Fiecărui cuvânt din vocabular îi este asociat un vector dens, de dimensiune fixă, notată cu `d_model`. În arhitectura Transformer originală, `d_model = 512`. Rezultatul este o secvență de vectori care păstrează ordinea cuvintelor din propoziția inițială.

Ce este embedder-ul?

- Embedder-ul este, din punct de vedere matematic, o **mapare liniară**, reprezentată printr-o matrice de embedding notată cu E.
- Această matrice are dimensiunea (d_model, $vocab_size$), unde `vocab_size` reprezintă numărul total de cuvinte din vocabular. Fiecare coloană a matricei corespunde unui cuvânt și definește vectorul său de embedding.
- După aplicarea embedder-ului, obținem o colecție de vectori de dimensiune `d_model`, unul pentru fiecare cuvânt din secvență. Această reprezentare este extrem de importantă, deoarece același format (vectori de dimensiune `d_model`) va fi utilizat în mod repetat în întregul Transformer: la intrarea encoderului, la ieșirea acestuia și în etapele ulterioare ale modelului.

Transformers – Embeddings

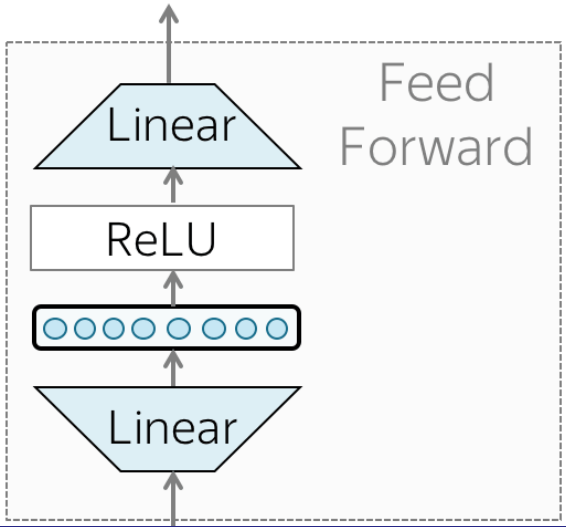
```
class Embeddings(nn.Module):
    def __init__(self, d_model, vocab):
        super(Embeddings, self).__init__()
        self.lut = nn.Embedding(vocab, d_model)
        self.d_model = d_model

    def forward(self, x):
        return self.lut(x) * math.sqrt(self.d_model)
```

- Această clasă implementează componenta de **embedding** din arhitectura Transformer. Obiectul `nn.Embedding` reprezintă o matrice de parametri învățabili care mapează fiecare token din vocabular într-un vector dens de dimensiune d_{model}. Intrarea `x` este o secvență de indici (token IDs), iar ieșirea este un tensor de dimensiune: $(\text{batch_size}, \text{seq_len}, d_{model})$.
- Embedding-urile sunt scalate cu $\sqrt{d_{model}}$ pentru a menține o magnitudine adecvată a activărilor și pentru a stabiliza antrenarea modelului, înainte de adăugarea informației de poziție (positional encoding).

Feed-Forward Blocks în Transformer

- Pe lângă mecanismele de atenție, **fiecare strat** din Transformer conține și un **feed-forward network (FFN)**.
- Acest FFN este alcătuit din:
  - două straturi liniare,
  - o funcție de activare **ReLU** între ele.
- Intuiție:
  - **Atenția**: *Privește alte tokenuri și adună informații.*
  - **FFN-ul**: *Procesează și rafinează informația obținută.*
- Fiecare token este procesat **independent** de același FFN, ceea ce crește capacitatea modelului de a învăța transformări complexe.

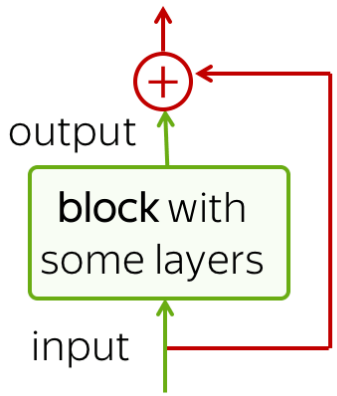


Residual Connections în Transformer

- **Residual connections** = se adaugă **intrarea** unui bloc la **ieșirea** sa:

$$\text{output} = \text{Block}(x) + x$$

- Roluri:
  - facilitează **fluxul gradientului**;
  - permit **antrenarea rețelelor foarte adânci**;
  - stabilizează reprezentările între straturi.
- În Transformer, apar după fiecare bloc:
  - **Self-Attention**;
  - **Feed-Forward Network (FFN)**.
- **Add & Norm**:
  - **Add** = conexiunea reziduală;
  - **Norm** = **LayerNorm** pentru stabilitate numerică.



Residual connection: add a block's input to its output

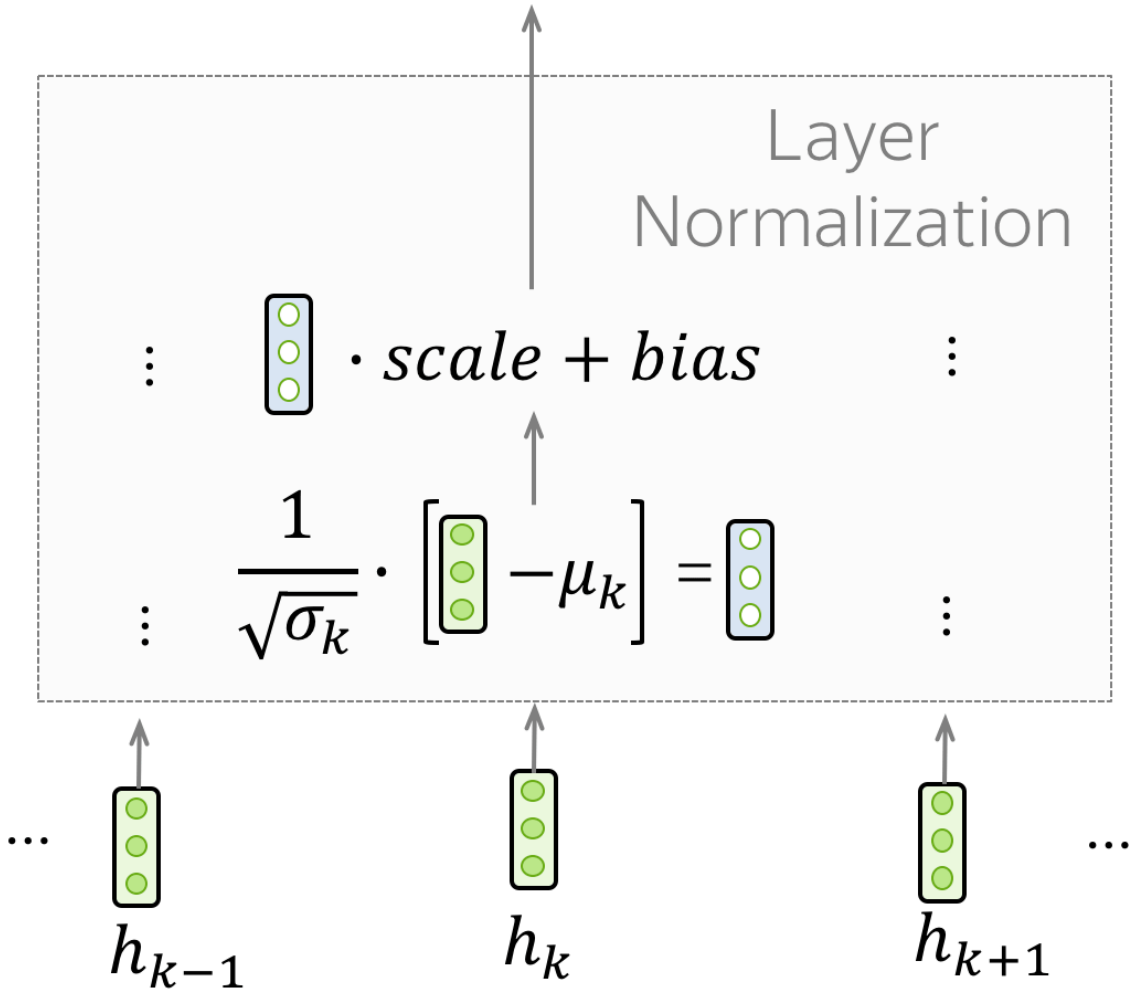
Layer Normalization în Transformer

- În blocul **Add & Norm**, partea de **Norm** reprezintă **Layer Normalization**.
- **LayerNorm** normalizează independent fiecare vector de reprezentare al fiecărui token:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma} + \beta$$

  unde:
  - μ, σ sunt media și deviația standard ale valorilor din vector;
  - γ, β sunt parametri **antrenabili** care rescalează semnalul.
- Scopuri:
  - stabilizează și accelerează **convergența**;
  - controlează *fluxul* informației între straturi;
  - previne explozia sau dispariția valorilor activărilor.
- În Transformer, LayerNorm se aplică asupra fiecărui **token** după:
  - blocul de **Self-Attention**;
  - blocul **Feed-Forward**.

Layer Normalization în Transformer

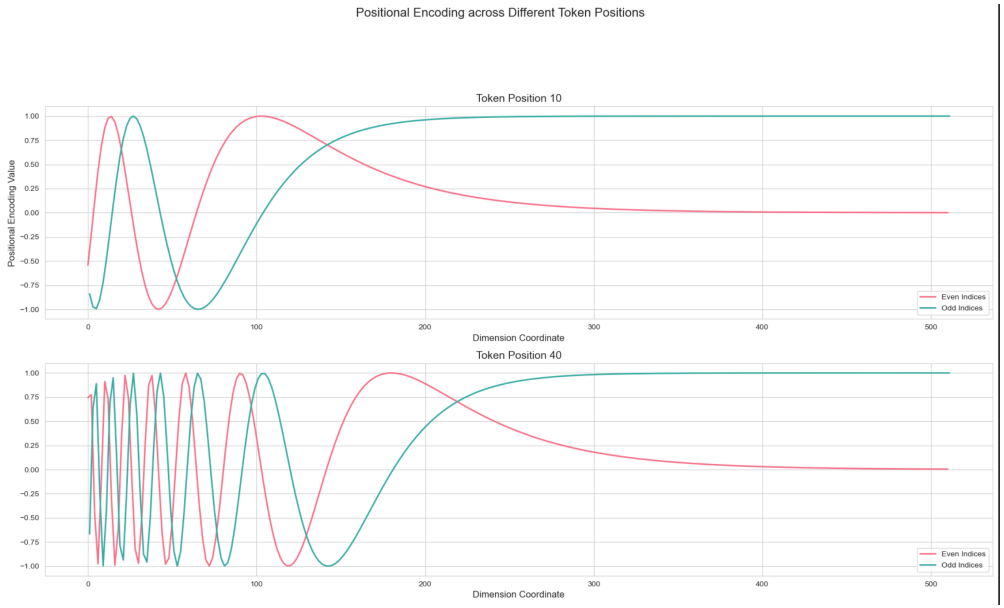


Transformers – Positional Encoding: motivație

- După etapa de embedding, fiecare cuvânt din secvență este reprezentat printr-un vector numeric. Totuși, aceste reprezentări **nu conțin nicio informație despre ordinea cuvintelor**. Dacă ar fi introduse direct în Transformer, propoziții precum *John loves Mary* și *Mary loves John* ar avea aceleași reprezentări, deși sensul lor este diferit.
- O soluție aparent simplă ar fi să adăugăm valori incrementale pentru fiecare poziție ($+1$, $+2$, $+3$, ...). Această abordare nu funcționează bine din mai multe motive. În primul rând, fiecare token este reprezentat într-un spațiu multidimensional ($d_{model} = 512$), iar o simplă incrementare scalară nu poate captura această complexitate. În al doilea rând, valorile mari ar afecta normalizarea și stabilitatea numerică a modelului. În plus, poziția absolută într-un text lung nu este relevantă; sensul unui cuvânt depinde mai degrabă de **poziția relativă** în propoziție.

Transformers – Positional Encoding: motivație

- Pentru a rezolva aceste probleme, autorii Transformer-ului introduc **Positional Encoding**, bazat pe funcții sinus și cosinus. Pentru fiecare poziție din secvență și pentru fiecare dimensiune a vectorului, se adaugă valori sinusoidale astfel încât dimensiunile pare folosesc sin, iar dimensiunile impare folosesc cos. Aceste funcții au frecvențe diferite, permițând modelului să distingă poziții apropiate și îndepărtate și să învețe relații de ordine în mod eficient.



Positional Encoding în Transformer

- Transformer-ul nu are recurență sau convoluție $\Rightarrow$ **nu cunoaște ordinea tokenilor**.
- Pentru a introduce ordinea, se folosesc două tipuri de embedding-uri:
  - **Token embeddings**
  - **Positional embeddings**
- Reprezentarea finală a unui token este:

$$\text{Embedding}(\textit{token}) + \text{PositionalEncoding}(\textit{pos})$$

- Autorii au observat că **positional encodings fixe** funcționează foarte bine.
- Pentru poziția *pos* și dimensiunea *i* a vectorului:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \qquad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- Fiecare dimensiune corespunde unei **sinusoide**, având lungimi de undă într-o progresie geometrică de la 2π la $10000 \cdot 2\pi$.

Positional Encoding în Transformer

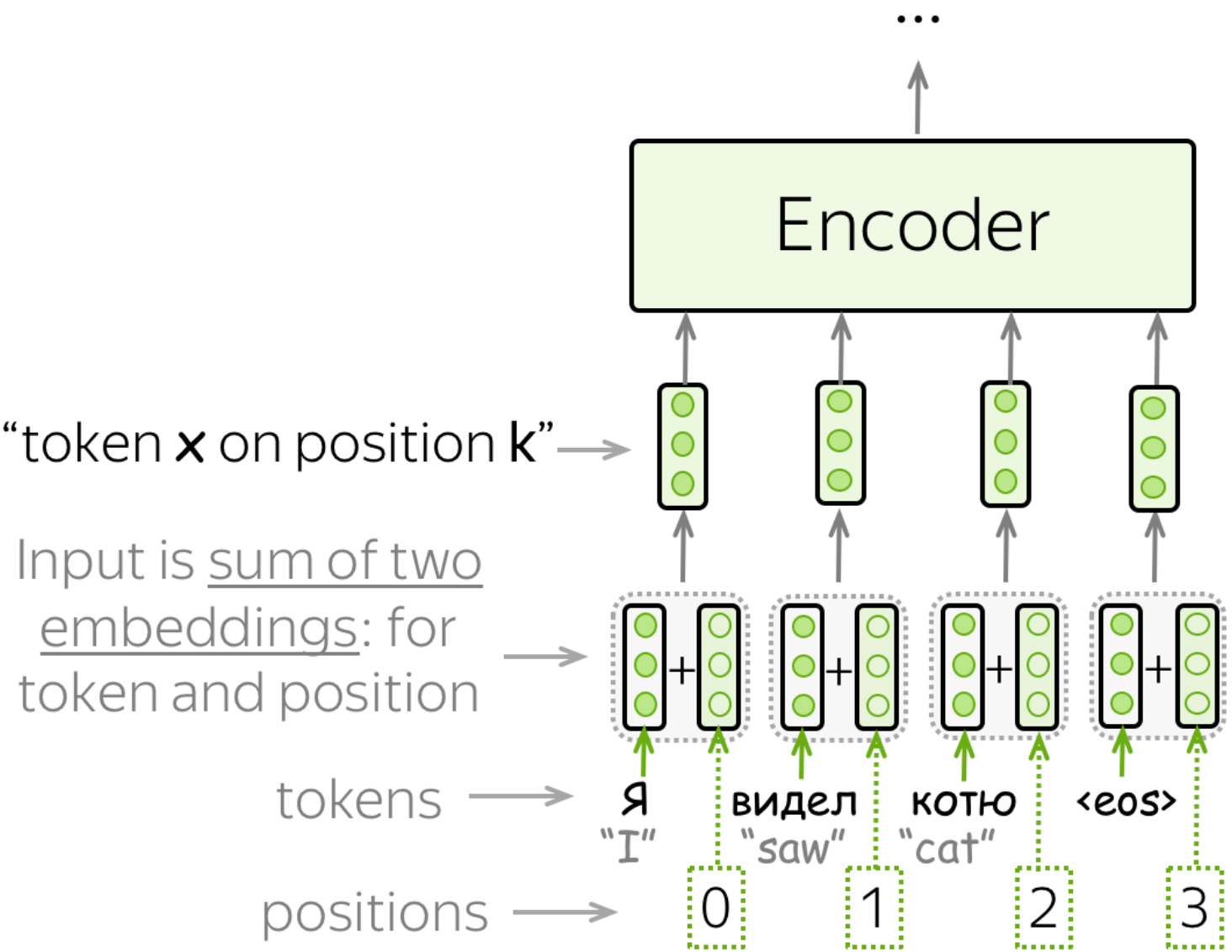
```
class PositionalEncoding(nn.Module):
    "Implement the PE function."

    def __init__(self, d_model, dropout, max_len=5000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

        # Compute the positional encodings once in log space.
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * -(math.log(10000.0) / d_model)
        )
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[:, : x.size(1)].requires_grad_(False)
        return self.dropout(x)
```

Positional Encoding în Transformer



Attention Mechanism – intuiție (Single Head)

Mecanismul de atenție reprezintă ideea centrală a lucrării *"Attention is All You Need"* și poate fi văzut ca un **mecanism de comunicare între tokeni**. Acesta permite modelului să acorde importanță diferită elementelor din secvența de intrare atunci când produce o reprezentare de ieșire.

La nivel intuitiv, atenția funcționează ca un **reflector (spotlight)**: părțile relevante ale intrării sunt iluminate mai puternic, în timp ce elementele mai puțin relevante primesc o atenție redusă. Prin acest mecanism, modelul poate captura relațiile semantice și funcționale dintre cuvintele unei propoziții.

Attention Mechanism – intuiție (Single Head)

1. **Rolul atenției într-o propoziție**
   - cuantifică relațiile dintre cuvintele dintr-o secvență;
   - identifică ce tokeni sunt relevanți pentru un token curent;
   - permite capturarea dependențelor atât locale, cât și pe termen lung.
2. **Exemplu: Propoziția** `Ea i-a dat cartea ei`
   - cuvântul `ei` are o conexiune puternică cu `cartea`, deoarece indică posesia;
   - are o conexiune moderată cu `Ea`, sugerând referirea la aceeași entitate;
   - are conexiuni mai slabe cu `i-a dat`.

```
def attention(query, key, value, mask=None, dropout=None):
    "Compute 'Scaled Dot Product Attention'"
    d_k = query.size(-1)
    scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)
    p_attn = scores.softmax(dim=-1)
    if dropout is not None:
        p_attn = dropout(p_attn)
    return torch.matmul(p_attn, value), p_attn
```

Attention Mechanism – intuiție (Single Head)

$q_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ $\quad d = 2$

$k_1 = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$, $k_2 = \begin{bmatrix} 1 \\ 1/4 \end{bmatrix}$

$v_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $v_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

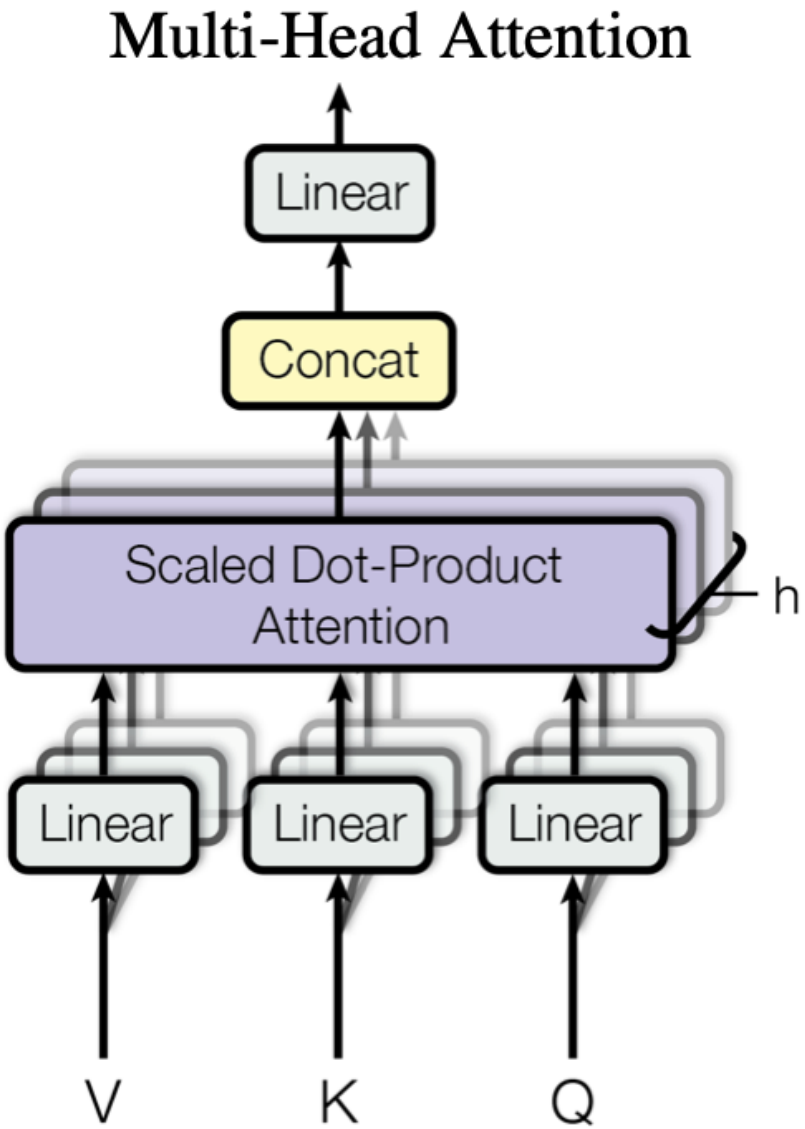
$$(\alpha_1, \alpha_2) = \text{Softmax}\left(\frac{3}{2}, \frac{1}{8}\right)$$

$$\alpha_1 = \frac{e^{\frac{3}{2}}}{e^{\frac{3}{2}} + e^{\frac{1}{8}}} \approx 0.8, \quad \alpha_2 = \frac{e^{\frac{1}{8}}}{e^{\frac{1}{8}} + e^{\frac{3}{2}}} \approx 0.2$$

$$\frac{\langle q_1, k_1 \rangle}{d} = \frac{3}{2} \qquad \frac{\langle q_1, k_2 \rangle}{d} = \frac{1}{8}$$

$$\text{Attention}(q_1, K, V) = \alpha_1 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} + \alpha_2 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.8 \end{bmatrix}$$

Attention Mechanism – intuiție (Multi Head)



Transformers – Multi-Head Attention

- În varianta de **single-head attention**, fiecare token formulează o singură interogare (*query*). Din cauza funcției *softmax*, atenția tinde să se concentreze puternic asupra unui singur token relevant, în timp ce contribuțiile celorlalți tokeni devin foarte mici. Totuși, în limbajul natural, un cuvânt este adesea relaționat simultan cu mai multe alte cuvinte din propoziție.
- Pentru a depăși această limitare, este introdus mecanismul **Multi-Head Attention**.
- Ideea principală este de a rula mecanismul de atenție în paralel de mai multe ori, permițând fiecărui token să pună **mai multe întrebări (queries)** în același timp. Fiecare *cap de atenție* poate învăța un tip diferit de relație (sintactică, semantică, de coreferință etc.).
- În arhitectura Transformer originală sunt utilizate **8 capete de atenție**. Rezultatele fiecărui cap sunt concatenate într-o singură matrice și apoi proiectate înapoi în spațiul de dimensiune d_{model}.
- Implementarea este relativ simplă, însă necesită o atenție deosebită la gestionarea dimensiunilor tensorilor.

Transformers – Multi-Head Attention

```python
class MultiHeadedAttention(nn.Module):
    def __init__(self, h, d_model, dropout=0.1):
        "Take in model size and number of heads."
        super(MultiHeadedAttention, self).__init__()
        assert d_model % h == 0
        # We assume d_v always equals d_k
        self.d_k = d_model // h
        self.h = h
        self.linears = clones(nn.Linear(d_model, d_model), 4)
        self.attn = None
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, query, key, value, mask=None):
        if mask is not None:
            # Same mask applied to all h heads.
            mask = mask.unsqueeze(1)
        nbatches = query.size(0)

        # 1) Do all the linear projections in batch from d_model => h x d_k
        query, key, value = [
            lin(x).view(nbatches, -1, self.h, self.d_k).transpose(1, 2)
            for lin, x in zip(self.linears, (query, key, value))
        ]

        # 2) Apply attention on all the projected vectors in batch.
        x, self.attn = attention(query, key, value, mask=mask, dropout=self.dropout)

        # 3) "Concat" using a view and apply a final linear.
        x = x.transpose(1, 2).contiguous().view(nbatches, -1, self.h * self.d_k)
        del query
        del key
        del value
        return self.linears[-1](x)
```

De ce Multi-head attention? (I)

- **Captează mai multe relații simultan** – Fiecare cap de atenție se concentrează pe diferite relații sau dependențe. De exemplu, un cap ar putea capta dependențe pe termen scurt (de exemplu, cuvinte adiacente), în timp ce un alt cap s-ar putea concentra pe relații pe termen lung (de exemplu, pronume care se referă la substantive îndepărtate).
- **Îmbunătățește puterea de reprezentare** – Prin analizarea simultană a mai multor aspecte ale intrării, atenția cu mai multe capete produce o reprezentare mai bogată și mai expresivă a secvenței de intrare.
- **Gestionează complexitatea în limbaj** – Limbajul natural are adesea structuri ierarhice și suprapuse. Atenția cu mai multe capete le poate intelege prin atribuirea diferitelor capete de atenție diferitelor structuri (de exemplu, sintaxă, semantică, co-referință).
- **Previne overfitting** – În loc să se bazeze pe un singur mecanism de atenție, modelul va avea capacitate de generalizare mai mare prin folosirea mai multor capete de atenție independente.

De ce Multi-head attention? (II)

- **Diversitatea focalizării** – Un mecanism de atenție cu un singur cap s-ar putea concentra prea îngust pe un aspect al intrării. Atenția cu mai multe capete permite modelului să învețe diverse șabloane, făcându-l mai robust și mai versatil.
- **Paralelizare** – Atenția cu mai multe capete împarte intrarea în diferite capete de atenție, făcând calculele mai eficiente și mai scalabile pentru date de dimensiuni mari
- **Precizie îmbunătățită** – Cu mai multe capete de atenție, modelul poate capta mai bine relațiile subtile și complexe în date, ceea ce duce la performanțe mai mari în taskuri precum traducerea sau răspunsul la întrebări.
- **Reducerea dimensionalității** – Fiecare cap operează într-un subspațiu de dimensiuni inferioare, făcând calculul mai eficient, captând în același timp întreaga complexitate atunci când capetele sunt combinate.

Asamblarea componentelor Transformerului

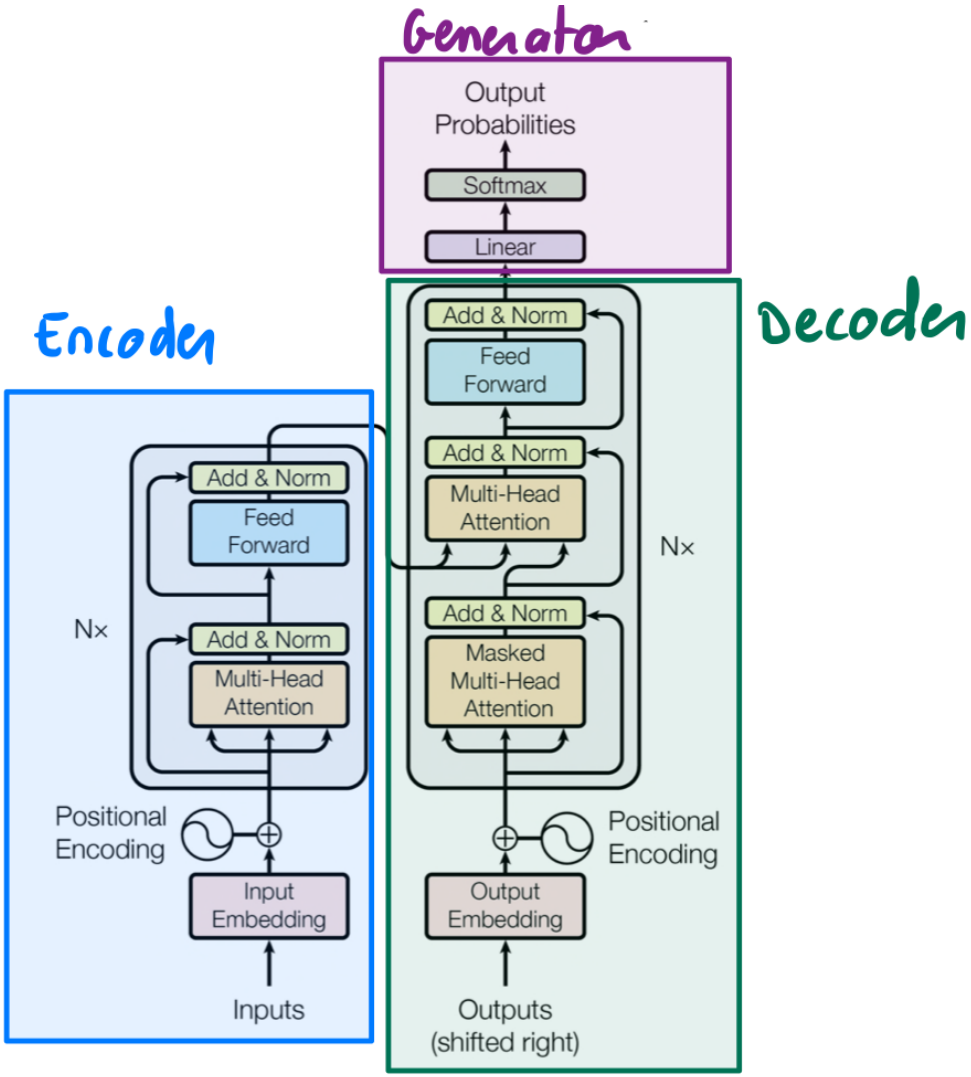


Figure 1: The Transformer - model architecture.

Transformers – Encoder, Decoder și Generator

Encoderul

- Scopul encoderului este de a transforma secvența de intrare într-o nouă secvență de vectori care captează esența informației inițiale.
- Această reprezentare este mai abstractă și mai informativă decât intrarea brută.
- Modele precum **BERT** folosesc exclusiv această componentă.

Decoderul

- Decoderul generează secvența de ieșire folosind informația produsă de encoder.
- Spre deosebire de un autoencoder clasic, decoderul din Transformer:
  - utilizează ieșirea encoderului;
  - ține cont și de tokenii generați anterior.
- Acest mecanism permite generarea secvențială coerentă a textului.

Generatorul

- Generatorul transformă un vector de ieșire într-un token concret.
- Vectorul este proiectat la dimensiunea vocabularului.
- Funcția *softmax* este utilizată pentru a selecta tokenul cu probabilitatea cea mai mare.

Implementare pentru Encoder–Decoder

```
class EncoderDecoder(nn.Module):
    """
    A standard Encoder-Decoder architecture. Base for this and many
    other models.
    """

    def __init__(self, encoder, decoder, src_embed, tgt_embed, generator):
        super(EncoderDecoder, self).__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.src_embed = src_embed
        self.tgt_embed = tgt_embed
        self.generator = generator

    def forward(self, src, tgt, src_mask, tgt_mask):
        "Take in and process masked src and target sequences."
        return self.decode(self.encode(src, src_mask), src_mask, tgt, tgt_mask)

    def encode(self, src, src_mask):
        return self.encoder(self.src_embed(src), src_mask)

    def decode(self, memory, src_mask, tgt, tgt_mask):
        return self.decoder(self.tgt_embed(tgt), memory, src_mask, tgt_mask)
```

Implementarea pentru Generator

```
class Generator(nn.Module):
    "Define standard linear + softmax generation step."

    def __init__(self, d_model, vocab):
        super(Generator, self).__init__()
        self.proj = nn.Linear(d_model, vocab)

    def forward(self, x):
        return log_softmax(self.proj(x), dim=-1)
```

Transformers – Stacking Encoders

Encoderul din arhitectura Transformer nu este alcătuit dintr-un singur strat, ci dintr-un **stack de N straturi identice**. În modelul Transformer original, numărul de straturi este $N = 6$. Fiecare strat procesează ieșirea stratului anterior, permițând modelului să construiască reprezentări din ce în ce mai abstracte.

Structura unui strat de Encoder

- Fiecare strat conține două blocuri principale (sublayers), fiecare urmat de:
  - o conexiune reziduală;
  - o normalizare de tip *LayerNorm*.

Transformers – Stacking Encoders

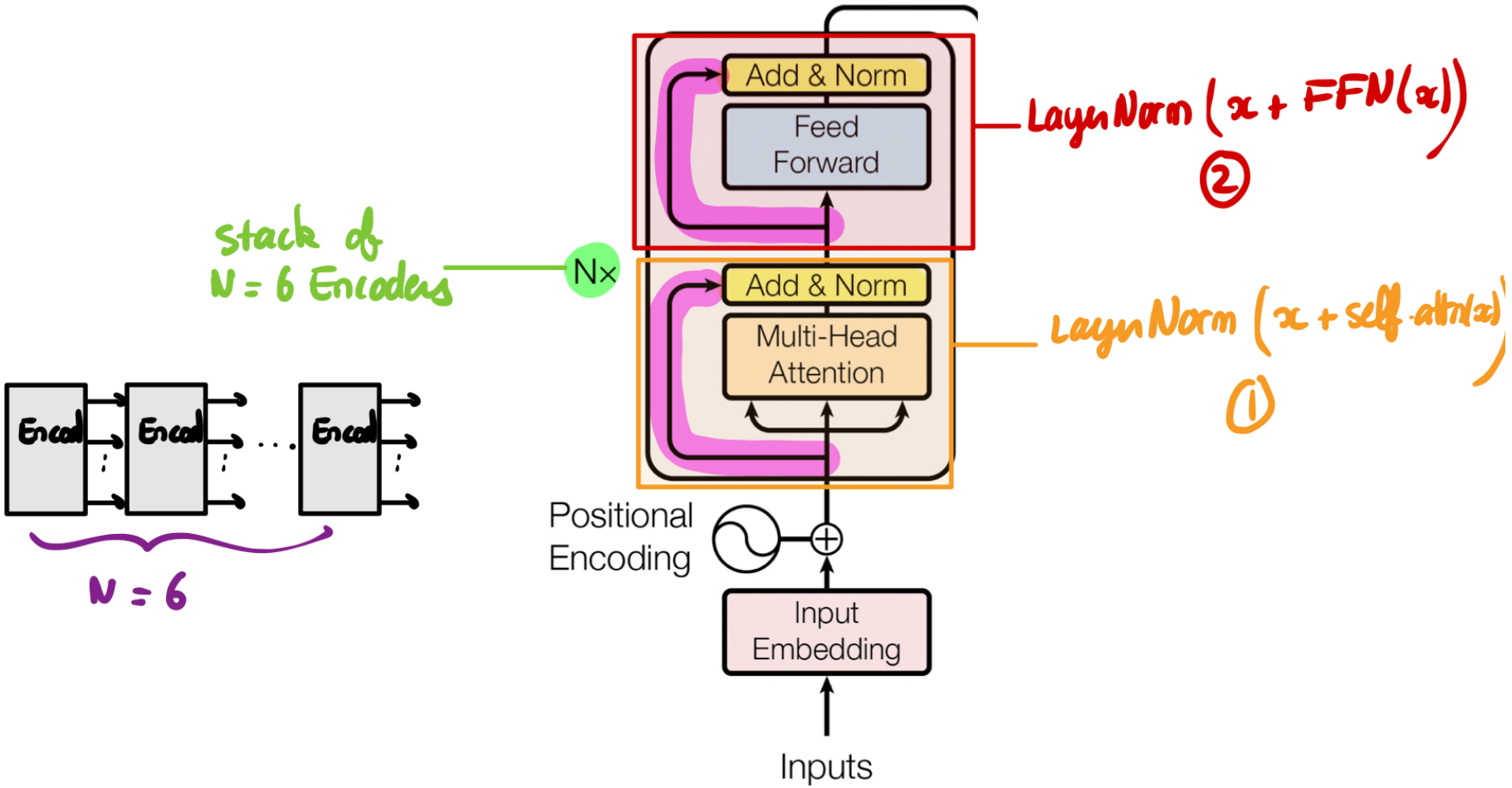
(1) Self-Attention

- Permite fiecărui token să țină cont de toți ceilalți tokeni din secvența de intrare.
- Ajută encoderul să identifice relațiile relevante dintre cuvinte.

(2) Feed-Forward Network

- O rețea neuronală complet conectată, aplicată independent fiecărei poziții.
- Transformă reprezentările obținute prin atenție într-un spațiu mai expresiv.

Transformers – Stacking Encoders



Implementare Encoder

```
class SublayerConnection(nn.Module):
    """
    A residual connection followed by a layer norm.
    Note for code simplicity the norm is first as opposed to last.
    """

    def __init__(self, size, dropout):
        super(SublayerConnection, self).__init__()
        self.norm = nn.LayerNorm(size)  # Use PyTorch's LayerNorm
        self.dropout = nn.Dropout(dropout)

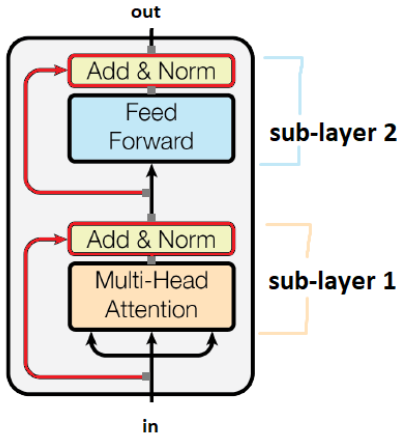
    def forward(self, x, sublayer):
        "Apply residual connection to any sublayer with the same size."
        return x + self.dropout(sublayer(self.norm(x)))
```

Implementare Encoder

```
class EncoderLayer(nn.Module):
    "Encoder is made up of self-attn and feed forward (defined below)"

    def __init__(self, size, self_attn, feed_forward, dropout):
        super(EncoderLayer, self).__init__()
        self.self_attn = self_attn
        self.feed_forward = feed_forward
        self.sublayer = clones(SublayerConnection(size, dropout), 2)
        self.size = size

    def forward(self, x, mask):
        # self attention, block 1
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, mask))
        # feed forward, block 2
        x = self.sublayer[1](x, self.feed_forward)
        return x
```



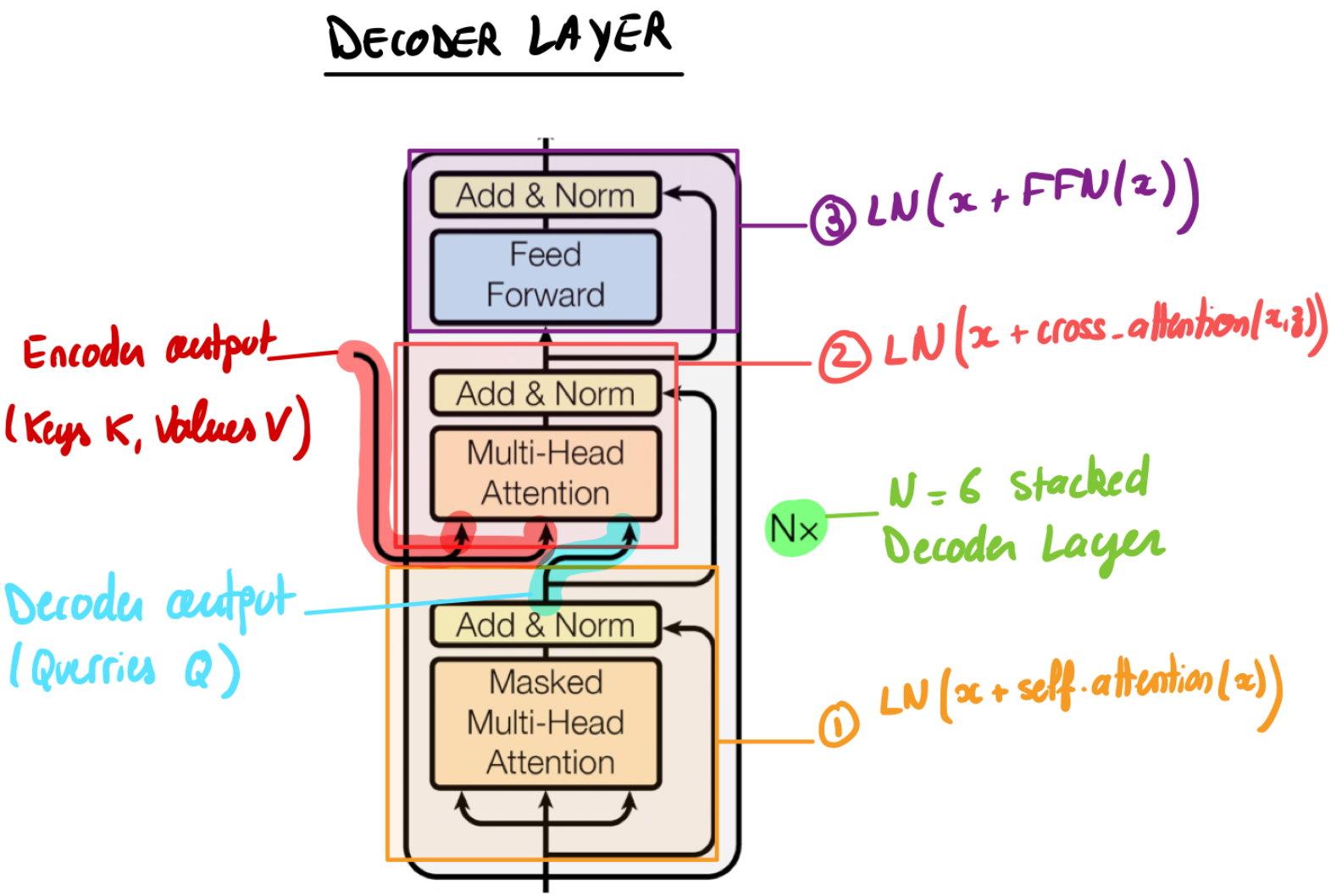
Implementare Encoder

```
def clones(module, N):
    "Produce N identical layers."
    return nn.ModuleList([copy.deepcopy(module) for _ in range(N)])

class Encoder(nn.Module):
    "Core encoder is a stack of N layers"
    def __init__(self, layer, N):
        super(Encoder, self).__init__()
        self.layers = clones(layer, N)
        self.norm = nn.LayerNorm(layer.size)
    def forward(self, x, mask):
        "Pass the input (and mask) through each layer in turn."
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)
```

Encoderul din arhitectura Transformer este responsabil de transformarea secvenței de intrare într-o reprezentare contextualizată, în care fiecare token încorporează informație despre toate ceilalți tokeni din secvență. Acesta este alcătuit dintr-un stack de straturi identice, fiecare strat conținând două subcomponente principale: mecanismul de self-attention și o rețea feed-forward aplicată pozițional. Fiecare subcomponentă este înconjurată de o conexiune reziduală și o normalizare de tip LayerNorm, ceea ce ajută la stabilizarea antrenării și la propagarea eficientă a gradientului. Prin aplicarea succesivă a acestor straturi, encoderul construiește reprezentări din ce în ce mai abstracte și mai bogate semantic, care pot fi ulterior utilizate de decoder sau de alte componente ale modelului.

Transformers – Decoderul



Transformers – Decoderul

La fel ca Encoderul, **Decoderul** din arhitectura Transformer este alcătuit dintr-un **stack de straturi identice**, numărul standard fiind $N = 6$ în modelul Transformer original. Fiecare strat procesează ieșirea stratului anterior și contribuie la construirea secvenței de ieșire.

- **Diferența esențială față de Encoder** constă în introducerea unui al treilea **sublayer**, care permite interacțiunea directă cu ieșirea encoderului: **Cross-Attention**.
- Sarcina decodorului este de a genera text.
- Spre deosebire de encoder, ieșirea decodorului este trimisă la un strat softmax pentru a calcula probabilitatea următorului cuvânt din secvență.

Structura unui strat de Decoder

- **(1) Self-Attention**: similar cu encoderul, toate componentele (Query, Key, Value) sunt generate din tokenii introduși în decoder. Acest mecanism permite modelului să țină cont de cuvintele generate anterior.
- **(2) Cross-Attention**: mecanismul prin care decoderul comunică cu encoderul. Interogările (Q) provin din ieșirea self-attention-ului, iar cheile și valorile (K, V) sunt generate din ieșirea encoderului. Astfel, generarea ține cont atât de contextul deja produs, cât și de informația din secvența de intrare.
- **(3) Feed-Forward Network**: identic cu cel din encoder, aplicat independent fiecărei poziții.

Implementare Decoder

```
class DecoderLayer(nn.Module):
    "Decoder is made of self-attn, src-attn, and feed forward (defined below)"

    def __init__(self, size, self_attn, src_attn, feed_forward, dropout):
        super(DecoderLayer, self).__init__()
        self.size = size
        self.self_attn = self_attn
        self.src_attn = src_attn
        self.feed_forward = feed_forward
        self.sublayer = clones(SublayerConnection(size, dropout), 3)

    def forward(self, x, memory, src_mask, tgt_mask):
        "Follow Figure for connections."
        m = memory
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, tgt_mask))
        # New sublayer (cross attention)
        x = self.sublayer[1](x, lambda x: self.src_attn(x, m, m, src_mask))
        return self.sublayer[2](x, self.feed_forward)
```

Implementare Decoder

```
class Decoder(nn.Module):
    "Generic N layer decoder with masking."

    def __init__(self, layer, N):
        super(Decoder, self).__init__()
        self.layers = clones(layer, N)
        self.norm = nn.LayerNorm(layer.size)

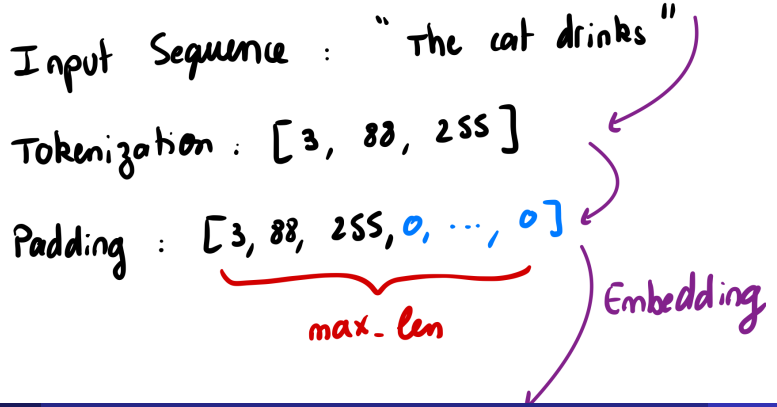
    def forward(self, x, memory, src_mask, tgt_mask):
        for layer in self.layers:
            x = layer(x, memory, src_mask, tgt_mask)
        return self.norm(x)
```

Proprietăți Decoder

- Decodorul este autoregresiv, ceea ce înseamnă că prezice valorile viitoare pe baza valorilor anterioare
- Decodorul prezice următorul token din secvență, uitându-se la ieșirea encoderului și la propria sa ieșire anterioară
- La fel ca la encoder, se adauga codificări poziționale la embeddingul cuvântului pentru a capta poziția token-urilor în propoziție.

Transformers – Padding

Într-un Transformer standard, există o lungime maximă a secvențelor, notată de obicei cu `max_len` (de exemplu, `max_len = 5000`). Aceasta reprezintă cea mai lungă secvență pe care modelul o poate procesa.
În practică, propozițiile au lungimi diferite. Pentru a putea procesa mai multe secvențe simultan (într-un batch), este necesar ca toate secvențele să aibă aceeași lungime. Pentru secvențele mai scurte, se utilizează **padding-ul**.
Padding-ul constă în adăugarea unor **tokeni speciali de umplere** (*padding tokens*) la finalul secvențelor mai scurte, astfel încât toate secvențele din batch să aibă aceeași dimensiune. Acești tokeni nu conțin informație semantică și trebuie ignorați în timpul calculului atenției.



Transformers – Masking

Masking-ul este un mecanism esențial care asigură faptul că, în timpul calculului atenției, anumiți tokeni sunt **ignorați**. Astfel, modelul se concentrează doar pe informația relevantă.

Situații principale de masking

- **Source Masking (src_mask)**:
  - este utilizat pentru a masca tokenii de padding;
  - previne acordarea de atenție unor tokeni fără semnificație semantică;
  - este aplicat atât în encoder, cât și în cross-attention.
- **Target Masking (tgt_mask) / Look-Ahead Masking**:
  - este utilizat în decoder în timpul generării secvențiale;
  - fiecare token poate vedea" doar tokenii anteriori;
  - previne accesul la tokenii viitori din secvență;
  - asigură o generare autoregresivă corectă a textului.

Transformers – Aplicarea măștii în Attention

După construirea măștii, aceasta este utilizată direct în calculul scorurilor de atenție. Mai exact, pentru pozițiile care trebuie ignorate (de exemplu tokenii de padding sau tokenii viitori), se adaugă o valoare foarte mică, conceptual echivalentă cu $-\infty$.

Prin adăugarea lui $-\infty$ la scorurile corespunzătoare acestor poziții, funcția *softmax* va atribui o probabilitate zero acelor tokeni. Astfel, ei nu vor contribui deloc la rezultatul final al atenției.

Intuiție:

- tokenii validați de mască rămân neschimbați;
- tokenii mascați primesc probabilitate zero după *softmax*;
- mecanismul de atenție se concentrează exclusiv pe tokenii relevanți.

Această tehnică este utilizată atât pentru mascarea tokenilor de padding, cât și pentru *causal masking* în decoder, asigurând o procesare corectă a secvențelor.

Transformers – Masking

"The Cat", $\begin{cases} seq_len = 2 \\ max_seq_len = 3 \end{cases}$

For query q_1, let's note $Score(i) = s_i = \frac{\langle q_1, k_i \rangle}{d}$

$s_3 \neq 0$, but it's an empty (padded) token

$$\alpha_1, \alpha_2, \alpha_3 = Softmax(s_1, s_2, s_3 + (-\infty))$$

$$\Rightarrow \alpha_3 = \frac{e^{s_3 - \infty}}{e^{s_1} + e^{s_2} + e^{s_3 - \infty}} = 0$$

Masking

- Deoarece decoderul încearcă să genereze secvența cuvânt cu cuvânt, se folosește o mască de anticipare pentru a indica ce intrări nu trebuie utilizate.
- De exemplu, atunci când se prezice cel de-al treilea token din propoziție, ar trebui să fie folosite doar simbolurile anterioare, adică primul și al doilea token.

	On	the	river	bank
On	0.7	0	0	0
the	0.2	0.9	0	0
river	0.4	0.1	0.8	0
bank	0.3	0.4	0.3	0.8

Masked multi-head attention

- Permite modelului să genereze secvențe de ieșire pas cu pas, asigurându-se că fiecare cuvânt dintr-o secvență poate vedea doar cuvintele generate anterior și nu pe cele viitoare.
- Prin mascarea viitorilor tokeni, se asigură că modelul este autoregresiv, adică predicțiile sunt făcute secvențial, folosind doar informațiile disponibile până la punctul current
- O mască este aplicată pentru a împiedica modelul să ia în considerare viitoarele poziții în secvență.
- Pentru fiecare poziție i din secvență, doar pozițiile anterioare lui i sunt vizibile.
- Masca este de obicei o matrice triunghiulară inferioară, unde elementele de deasupra diagonalei sunt setate la o valoare mare negativă după aplicarea funcției softmax, astfel încât să fie ignorate.

De ce Masked multi-head attention?

- Mascarea este utilizată pentru a păstra caracterul cauzal al modelului, mai ales în timpul generării secvențiale.
- Acest lucru este important din mai multe motive:
  1. **Previne scurgerile de informații viitoare** – atunci când modelul generează un token nou, acesta nu trebuie să "vadă" viitoarele token-uri din secvență, deoarece acestea încă nu au fost generate. De exemplu, dacă modelul completează fraza `Pisica mănâncă...`, acesta nu trebuie să știe deja că urmează `pește`.
  2. **Asigură coerența temporală** – mascarea asigură că modelul procesează token-urile în ordine cronologică, așa cum ar face un om care scrie sau citește.
  3. **Este necesară în timpul antrenării pentru sarcini de generare** – in timpul antrenării, modelul are acces la întreaga frază. Mascarea este necesară pentru a-l învăța să genereze text doar pe baza tokenilor anteriori, simulând modul în care va fi utilizat în timpul inferenței

Transformers – Feed-Forward Network (FFN)

Feed-Forward Network (FFN) din Transformer nu este o singură operație, ci un **lanț de transformări liniare și non-liniare** aplicate fiecărei poziții din secvență.

Structura FFN:

- Conține două straturi liniare.
- Datele de intrare, de dimensiune `d_model = 512`, sunt mai întâi mapate într-un spațiu mai mare, `d_ff = 2048`.
- Ulterior, datele sunt transformate înapoi la dimensiunea inițială `d_model = 512`.

Intuiție:

- Această expansiune" temporară permite rețelei să învețe relații mai complexe între caracteristicile vectorului.
- Compresia" înapoi la dimensiunea inițială menține consistența cu restul modelului și dimensiunea embedding-urilor.

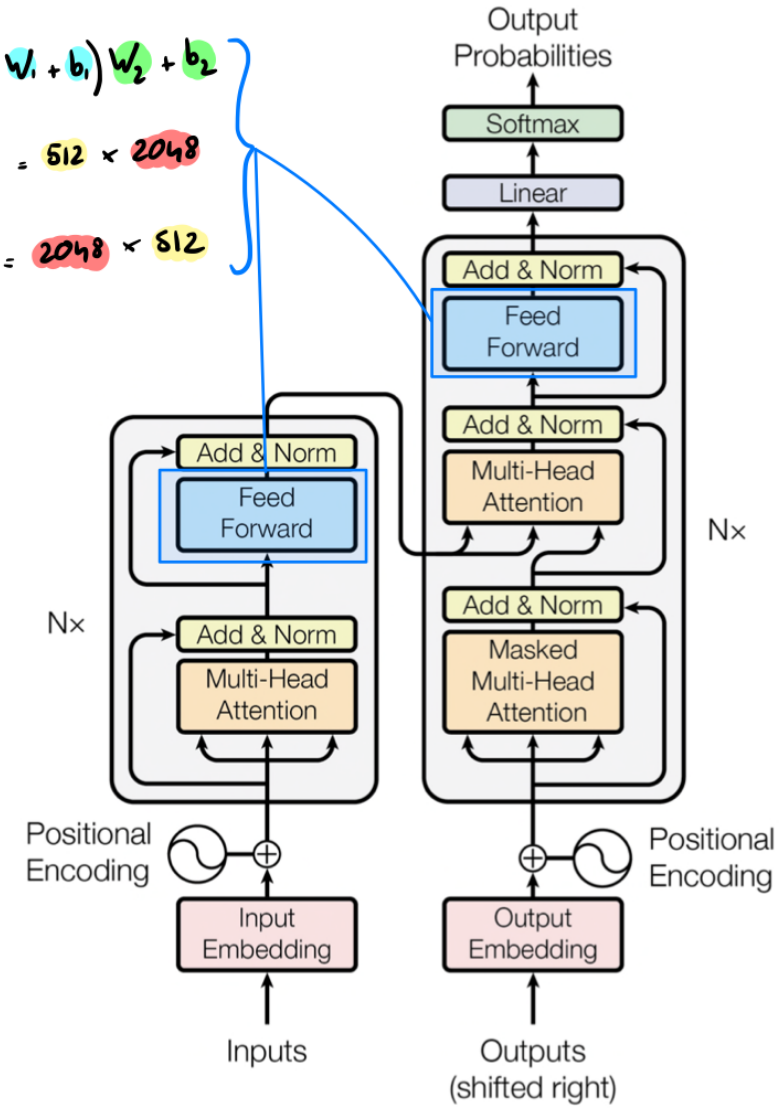
Astfel, FFN acționează ca un bloc de procesare poziționlă independentă, complementând mecanismul de atenție pentru a construi reprezentări mai bogate.

Transformers – Feed-Forward Network (FFN)

$$FFN(x) = ReLU\left(x W_1 + b_1\right) W_2 + b_2$$

W_1 - $d_{model} \times d_{ff} = 512 \times 2048$

W_2 - $d_{ff} \times d_{model} = 2048 \times 512$



Transformers – Feed-Forward Network (FFN)

```
class PositionwiseFeedForward(nn.Module):
    "Implements FFN equation."

    def __init__(self, d_model, d_ff, dropout=0.1):
        super(PositionwiseFeedForward, self).__init__()
        self.w_1 = nn.Linear(d_model, d_ff)
        self.w_2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        return self.w_2(self.dropout(self.w_1(x).relu()))
```

Transformers – Concluzie

Succesul și popularitatea excepțională a modelului Transformer se datorează mai multor factori cheie:

- **Flexibilitate:** Modelele Transformer pot procesa orice secvență de vectori. Acești vectori pot fi embeddings pentru cuvinte, patch-uri din imagini sau segmente audio vectorizate, ceea ce face modelul foarte adaptabil la diferite domenii.
- **Generalitate:** Cu un *inductive bias* minim, modelele Transformer pot captura tipare complexe și nuanțate în date, permițând o învățare și generalizare mai eficientă.
- **Viteză și eficiență:** Concepute pentru procesare paralelă, modelele Transformer valorifică puterea computațională a GPU-urilor, ceea ce accelerează antrenarea și inferența.

Aceste caracteristici fac din Transformer un model extrem de puternic și versatil, utilizat pe scară largă în NLP, Computer Vision și Audio.

Ce sunt Large Language Models (LLM)?

- **Large Language Models (LLM)** sunt modele neurale specializate în procesarea limbajului natural.
- Sunt antrenate pe volume foarte mari de text (cărți, articole, cod, pagini web).
- Scopul lor este să **înțeleagă** și să **genereze** limbaj uman coerent.

Cum funcționează LLM-urile?

- Se bazează pe arhitectura **Transformer**.
- Învață relații statistice dintre cuvinte și contexte.
- Generează text prezicând următorul cuvânt pe baza contextului anterior.
- Nu *înțeleg* ca oamenii, ci operează pe tipare din date.

Aplicații ale Large Language Models

- Chatboți și asistenți virtuali
- Traducere automată
- Rezumarea documentelor
- Generare de cod și asistență pentru programare
- Analiza sentimentelor și a textelor

A Survey of Large Language Models

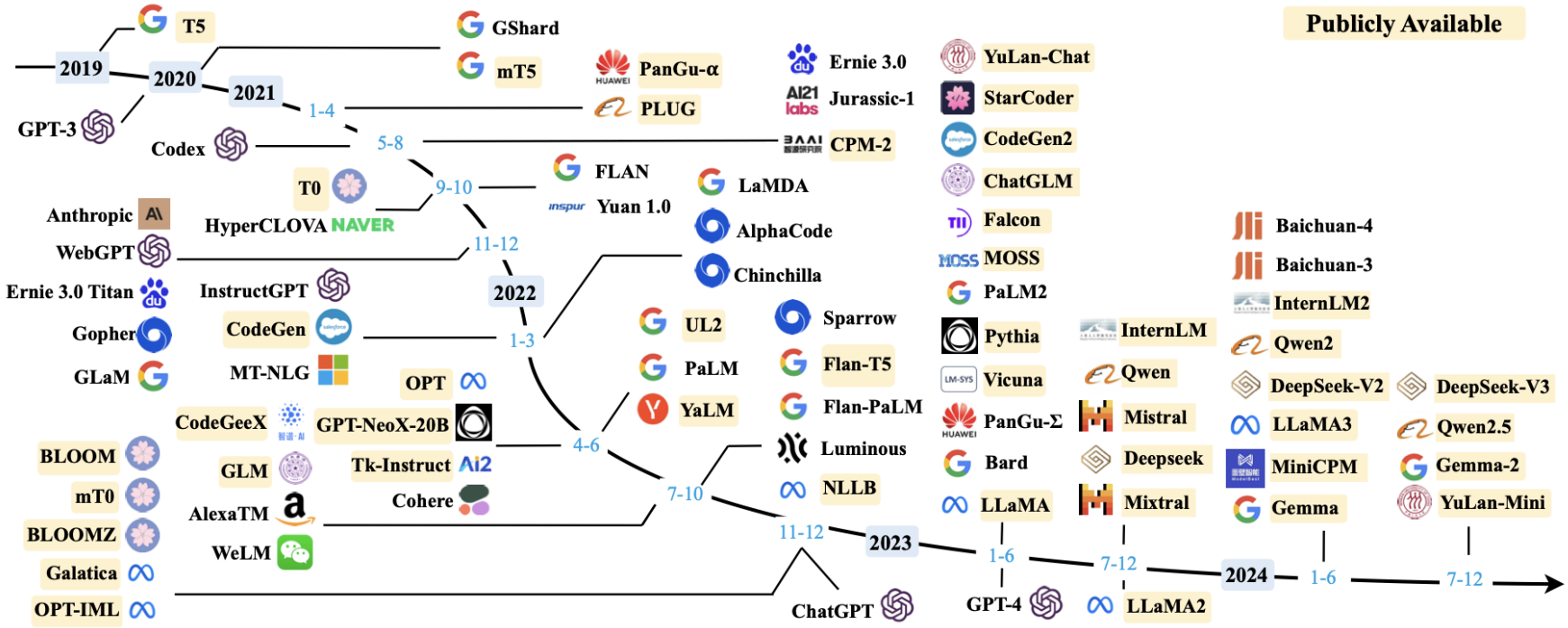


Fig. 3: A timeline of existing large language models (having a size larger than 10B) in recent years. The timeline was established mainly according to the release date (*e.g.*, the submission date to arXiv) of the technical paper for a model. If there was no corresponding paper, we set the date of a model as the earliest time of its public release or announcement. We mark the LLMs with publicly available model checkpoints in yellow color. Due to the space limit of the figure, we only include the LLMs with publicly reported evaluation results.

Sursă: https://arxiv.org/pdf/2303.18223

A Survey of Large Language Models

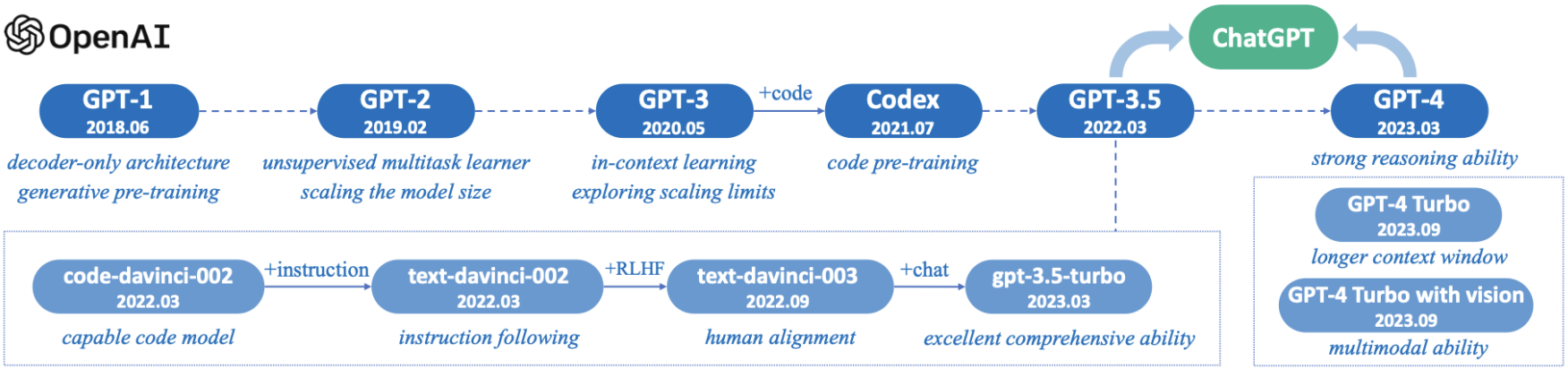
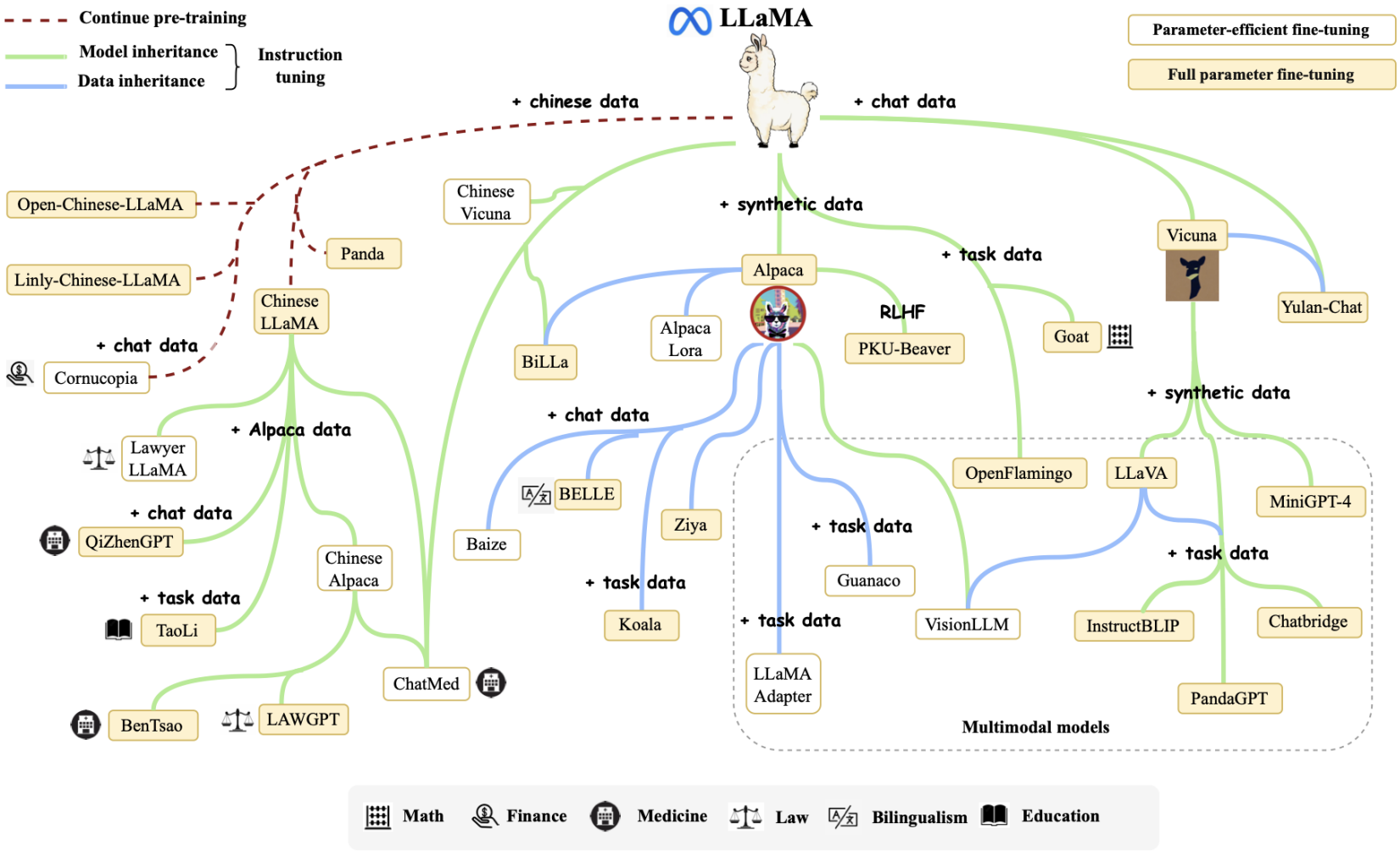


Fig. 4: A brief illustration for the technical evolution of GPT-series models. We plot this figure mainly based on the papers, blog articles and official APIs from OpenAI. Here, *solid lines* denote that there exists an explicit evidence (*e.g.*, the official statement that a new model is developed based on a base model) on the evolution path between two models, while *dashed lines* denote a relatively weaker evolution relation.

Sursă: https://arxiv.org/pdf/2303.18223

A Survey of Large Language Models



Sursă: https://arxiv.org/pdf/2303.18223

Referințe și resurse

- **Surse teoretice principale:**
  - Vaswani et al., *Attention is All You Need* (2017)
  - Mikolov et al., *Efficient Estimation of Word Representations in Vector Space* (Word2Vec)
  - Peters et al., *Deep Contextualized Word Representations* (ELMo)
  - Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers*
- **Resurse vizuale și animații:**
  - Ilustrațiile și animațiile din prezentare sunt preluate de pe: `https://lena-voita.github.io/`
- **Vizualizări interactive ale modelelor LLM:**
  - `https://bbycroft.net/llm`